AI AGENTS, MCP & APIS FOR PMS AND BAS · LESSON 7 OF 11

OpenAPI and Swagger: Describing APIs So Machines and People Can Use Them

Read and write an OpenAPI description of a REST API, covering paths, parameters, request bodies, responses, schemas and security. See how the same file drives documentation, client code and AI tool definitions, and why it must be kept in sync with the code.

Gap-fill lesson · ✦ 2026 · Intermediate · ⏱ 90 min

By the end of this lesson you can

- Explain what OpenAPI and Swagger are and who reads each file
- Describe paths, methods, parameters, request bodies and responses in OpenAPI
- Define a reusable schema and reference it
- Keep an API description in sync with the implementation

Why an API needs a written description

An API is a contract: a list of addresses, the data each one accepts, and what it returns. If the contract lives only in someone's head, integrations break and AI tools guess at parameters. An **OpenAPI** file is that contract written in a standard format (YAML or JSON). **Swagger** is the set of tools that turn the file into interactive documentation (Swagger UI).

CASE STUDY **A new integration in one afternoon**

A partner wants to pull order status from your system. Without a description, an engineer reads the code to find the endpoint, the parameters and the error format. With an OpenAPI file, the partner's developer opens the documentation, tries the call in the browser, and generates a client library in minutes.

The anatomy of an OpenAPI file

YAML

```yaml
openapi: 3.0.3
info:
  title: Orders API
  version: 1.0.0
servers:
  - url: https://api.example.com/v1
paths:
  /orders:
    get:
      summary: List orders
      parameters:
        - name: status
          in: query
          required: false
          schema: {type: string, enum: [open, shipped, cancelled]}
      responses:
        "200":
          description: A list of orders
          content:
            application/json:
              schema:
                type: array
                items: {$ref: "#/components/schemas/Order"}
components:
  schemas:
    Order:
      type: object
      required: [id, status]
      properties:
        id: {type: string, example: "ORD-1001"}
        status: {type: string, enum: [open, shipped, cancelled]}
        total: {type: number, example: 2499.0}
```

Read it from the top:

Section	Meaning
`info`	Name and version of the API
`servers`	Base address
`paths`	Each address and its methods (`get` , `post` , `put` , `delete`)
`parameters`	Inputs in the URL (`query` , `path` , `header`)
`requestBody`	Data sent with `post` or `put`
`responses`	Each possible status code and its body
`components/schemas`	Reusable data shapes, referenced with `$ref`

The example describes `GET /orders?status=open`, which is the sample in the gap audit: an optional status filter and a list of orders as the response.

Step 1: describe a write operation

A `POST` creates something, so it has a request body and a response for success and for errors.

YAML

```yaml
  /orders/{id}/cancel:
    post:
      summary: Cancel an open order
      parameters:
        - {name: id, in: path, required: true, schema: {type: string}}
      requestBody:
        required: true
        content:
          application/json:
            schema:
              type: object
              required: [reason]
              properties:
                reason: {type: string, maxLength: 200}
      responses:
        "200": {description: Order cancelled}
        "404": {description: No order with that id}
        "409": {description: Order already shipped and cannot be cancelled}
```

The `409` response documents a business rule (you cannot cancel a shipped order). Documenting error responses is as important as the success case, because integrators write code for both.

Step 2: validate the file

Tools check the file for mistakes before anyone relies on it. Common options include the `openapi-spec-validator` Python package and the online Swagger Editor, which shows errors as you type. Validate in your continuous-integration pipeline so that a broken description cannot be merged.

Python

```python
# Shown as code, not executed (needs the package installed):
# pip install openapi-spec-validator
from openapi_spec_validator import validate
import yaml

with open("orders_api.yaml") as f:
    spec = yaml.safe_load(f)
validate(spec)          # raises an error if the file breaks the OpenAPI rules
```

Step 3: one file, several uses

The same file can drive:

- **Documentation** for partners and internal developers (Swagger UI, Redoc).
- **Client code** generated for several languages.
- **Mock servers** so front-end teams can work before the back end exists.
- **Tool definitions for AI agents**. An agent framework or a model's tool-calling feature needs a name, a purpose and typed parameters for each tool. An OpenAPI operation already contains those, so it can be converted into a tool definition.

::::tip Describe the operations an AI tool should expose Not every endpoint should be an AI tool. Expose the read operations first, and describe each one with a clear `summary` and parameter descriptions, because the model reads those words to decide when to call the tool. ::::

Step 4: keep the description in sync

The most common failure is a description that no longer matches the code. Three habits prevent it:

1. Generate the description from the code where your framework supports it (many web frameworks produce OpenAPI automatically), then review it.
2. Run a contract test: call each documented endpoint in a test and check the response against its schema.
3. Version the API. A breaking change (renaming a field, removing a status) goes into a new version number, not a silent edit.

::::warn A specification that lies is worse than none If the documentation says a field exists and the code does not return it, integrators build broken software and blame you. Treat the description as code: review it in pull requests and test it. ::::

INTERVIEW CORNER

What makes an API description useful?

It is complete (every path, parameter and status code that can happen), precise (types, required fields and enum values), accurate (tested against the real service), and readable (clear summaries and examples). It is also versioned, so integrators know when something has changed.

::::key What to remember - OpenAPI is a standard, machine-readable description of a REST API; Swagger UI shows it as documentation. - Describe paths, parameters, request bodies, responses (including errors) and reusable schemas. - The same file serves documentation, client generation, mocks and AI tool definitions. - Validate it and test it against the service, or it drifts out of date. ::::

Quick quiz

5 questions · tap an answer to check it

1. In an OpenAPI file, where are reusable data shapes defined?

A In `servers`

B In `info`

C In `components/schemas`, referenced with `$ref`

D In each response body only

Answer: C. Components hold shared definitions, and `$ref` points to them.

2. Which location holds the `status` value in `GET /orders?status=open`?

A path

B header

C query

D body

Answer: C. Values after the question mark in the URL are query parameters.

3. Why document a 409 response for a cancel operation?

A It is required for every API

B It tells integrators about a business rule they must handle

C It makes the server faster

D It replaces the 200 response

Answer: B. Integrators write code for error cases as well as success.

4. What is a common cause of broken API documentation?

A The file is in YAML

B The description is not kept in sync with the code

C Swagger UI is installed

D Using JSON instead of YAML

Answer: B. Drift between the description and the implementation is the usual failure.

5. Why describe read operations first when exposing an API as AI tools?

A They are cheaper to write

B They are lower risk, and the model can use their clear descriptions to decide when to call them

C Write operations are not allowed in OpenAPI

D Models cannot call POST

Answer: B. Start with safe, read-only operations and add writes with approval.

EXERCISE **Describe two endpoints**

Write an OpenAPI 3.0 file for two endpoints of an application you know (or of the orders example): one `GET` with a query parameter and a response schema, and one `POST` with a request body and at least two error responses. Validate it with an online editor or the validator package, and list two things the code would need to return to match your description.

Show solution (worked answer)

Hints: use `components/schemas` for the main object and reference it in both operations. Give each parameter a `description`, because it is the text an AI tool would read. Check that the `required` list matches the fields your code always sends.

Practise more online

Free, trusted places to test yourself on this topic. Links open the original site in a new tab.